arXiv:2106.12895v1 [cs.LG] 15 Jun 2021

rSoccer: A Framework for Studying Reinforcement Learning in Small and Very Small Size Robot Soccer

Felipe B. Martins, Mateus G. Machado, Hansenclever F. Bassani, Pedro H. M. Braga, and Edna S. Barros

Centro de Informática - Universidade Federal de Pernambuco, Av. Jornalista Anibal Fernandes, s/n - CDU 50.740-560, Recife, PE, Brazil.
{fbm2, mgm4, hfb, phmb4, ensb}@cin.ufpe.br

Abstract. Reinforcement learning is an active research area with a vast number of applications in robotics, and the RoboCup competition is an interesting environment for studying and evaluating reinforcement learning methods. A known difficulty in applying reinforcement learning to robotics is the high number of experience samples required, being the use of simulated environments for training the agents followed by transfer learning to real-world (sim-to-real) a viable path. This article introduces an open-source simulator for the IEEE Very Small Size Soccer and the Small Size League optimized for reinforcement learning experiments. We also propose a framework for creating OpenAI Gym environments with a set of benchmarks tasks for evaluating single-agent and multi-agent robot soccer skills. We then demonstrate the learning capabilities of two state-of-the-art reinforcement learning methods as well as their limitations in certain scenarios introduced in this framework. We believe this will make it easier for more teams to compete in these categories using end-to-end reinforcement learning approaches and further develop this research area.

Keywords: Reinforcement Learning · OpenAI Gym · Continuous Control · Robot Soccer · Simulation

1 Introduction

Reinforcement Learning (RL) [22], in conjunction with the machine learning field, has obtained interesting results in progressively more complex decision-making competitive scenarios, such as learning how to play Atari games [25], Go [21], and Starcraft 2 [24], achieving or even surpassing human-level performance. In robotics, RL showed promising results in simulated and real-world environments, including approaches for motion planning, optimization, grasping, manipulation, and control [1, 5, 11].

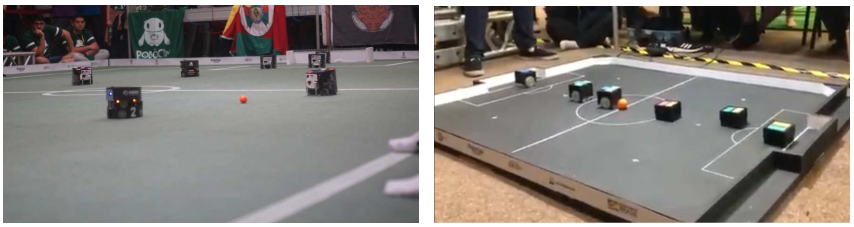
Robot soccer competitions are an exciting field for researching and validating RL usage, as it involves robotic systems capable of tackling challenging sequential decision-making problems in a cooperative and competitive scenario [8].

Developing those systems can be very hard using traditional methods in which hard-coded behaviors need to foresee a multitude of possibilities in an unpredictable game such as soccer.

In the RoboCup *Small Size League* (SSL) competition, Fig. 1(a), teams with up to eleven omnidirectional mobile robots compete against each other to score goals within a set of complex rules, such as limiting how far a robot can move with the ball, therefore requiring explicit cooperation. Previous works in this setting have successfully learned specific skills, such as moving to the ball, kicking, and defending penalties by using RL control approaches [26–28]. However, those works did not make the learning environment available, which can hinder reproducibility.

On the other hand, achieving an end-to-end control policy capable of cooperating on a robot soccer match is still an open problem, requiring even further research and development as the league evolves. Similarly, training a single policy capable of controlling a complete SSL team is also challenging, as the total number of control actions increases with the number of robots.

The *IEEE Very Small Size Soccer* (VSSS) competition, Fig. 1(b), compared to the SSL, establishes teams with three robots each, with a smaller field and robot sizes. The robot hardware does not have ball dribbling and kicking capabilities, and a match does not require explicit cooperation, by the rules. Although the differential drive robots used pose a more challenging path planning, the league can be seen as a simplified version of the SSL. In this domain, earlier work applied RL for learning specific skills [6]. And more recently, Bassani et al. [3] achieved 4th place in an international VSSS competition, using an end-to-end learned control without explicit cooperation and made the learning environments publicly available. Still, they do not support the SSL setting and are not easily adaptable for different scenarios.



(a) SSL (b) IEEE VSSS

Fig. 1. Competition environments of SSL and IEEE VSS.

We consider that by supplying a simpler path towards creating and using RL SSL environments, the results achieved by Bassani et al. [3] can be replicated on the SSL competition and be further used to encourage RL approaches in the SSL context. In summary, the contributions of the present work are the following:

1. An open-source framework following the OpenAI Gym [4] standards for developing robot soccer RL environments, modeling multi-agent tasks in competitive and cooperative scenarios;
2. An open-source SSL and VSSS robot soccer simulator, adapted from the grSim Simulator [16], focused on RL use;
3. A set of eight benchmark learning environments with a focus on reproducibility, for evaluating RL algorithms in robot soccer tasks, including four tasks based on the RoboCup SSL 2021 hardware challenges.

The rest of this article is organized as follows: Section 2 presents related work on robot soccer simulators and environments. Section 3 describes the proposed framework. Section 4 introduces a set of benchmark environments created using the proposed framework. Section 5 presents the results, and finally, Section 6 draws the conclusions and suggests future work.

2 Related Works

There is a large variety of RL environments and frameworks on the literature which aim at allowing the easy reproduction of state-of-the-art RL algorithms results, such as the OpenAI Gym [4]. However, the existing robot soccer environments lack the needed characteristics, such as extensibility to different scenarios, proper real-world robot simulation, and hard to reproduce results. Therefore, they do not apply to the RoboCup categories. These issues are discussed as follows.

Suitable frameworks. There are frameworks for simulating soccer matches such as the RoboCup's Soccer 2D [9] and The Google Research Football Environment [10]. However, the actions defined are too high level. The DeepMind MuJoCo Multi-Agent Soccer Environment [13] define low-level actions such as accelerating and rotating the body, but it is not related to a real robot soccer league. Bassani, *et. al* [3] proposed an framework for the VSSS setting, but it does not enable the creation of new scenarios. Although Robocup's Soccer 3D provides a low-level action and believable environment, there is no framework that enables the creation of scenarios.

Simulator's purpose. There are well known simulators for robot soccer competitions such as SSL [16] and VSSS [15, 19]. They provide a real-time simulated environment with a rich graphical interface for developing robot soccer algorithms. However, the preferences for RL are simulation speed and synchronous communication.

Reproducibility issues. Previous work achieved interesting results using RL in robot soccer competition settings [6, 20, 26]. But they do not describe the environments and simulators used nor made them openly available, coupled with a lack of clearly defined tasks and availability of stable baseline implementations of robot soccer agents, poses several issues to the advancement of research in this field.

3 rSoccer Gym Framework

The proposed framework[1] is a tool for creating robot soccer environments ranging from simple single-agent tasks to complex multi-robot competitive cooperative scenarios.

It is defined by three modules: **simulator**, **environment**, and **render**. The simulator module describes the physics simulation. The environment module is designed to receive the agent action, communicate with the other modules, and return the new observations and rewards. The render module does the environment visualization. Fig. 2 illustrates the modules architecture. A set of data structures labeled **entities** are defined to enable a common communication between modules for every environment. The following subsections describe these modules and *entities*.

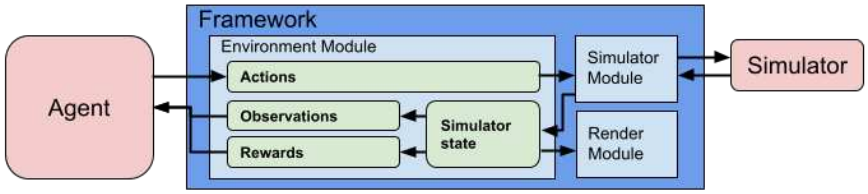


Fig. 2. Framework modules architecture

3.1 Entities

The *entities* structures are standardized for consistency by defining positional values using the field center as a reference point. The units conform to the International System of Units (SI), except for robot angular position and speed values which are in degrees. The following *entities* are defined:

- *Ball*: Contains the ball position and velocity values and is used both to read the current state or to set the initial position;
- *Robot*: Contains a robot identification, flags, position, velocity, and wheel desired speed values. Used to read the current state, to set the initial position, or to send control commands;
- *Frame*: Contains a *Ball entity* and *Robot entities* for each robot in the environment, structured in a way that each robot is easily indexable by team color and id. Used to store the complete state of the simulation;
- *Field*: Contains specifications of the simulation values, such as the field and robot geometry and parameters.

[1] Code available at `https://github.com/robocin/rSoccer`

3.2 Simulator Module

The simulator module carries out the environment physics calculations. It communicates directly with the environment module, receiving actions and returning the simulation state.

For physics calculations, we developed the rSim[2] simulator specially for RL. It was based on the grSim simulator [16], due to its reliable physical simulation, with the following modifications:

- Removal of graphical interfaces to increase performance, reduce memory usage, and ease server deployment on headless servers;
- Synchronous operation for more consistent training results as in an asynchronous setting the synchronization between agents and simulator may depend on hardware performance;
- Support for a different number of robots in each team, to enable more environment possibilities;
- Split simulated objects collision spaces to create separate collision groups;
- Added motor speed constraints matching real-world observations;
- Enable cylinder collision, removing the dummy collision object to reduce the total number of simulated bodies;
- Defined direct simulator calls in Python for fast communication. Enabling the instantiation of multiple simulators without the need to manage network communication ports.

Although the simulator is external to the framework, the simulator module abstracts its interface. Table 1 presents the rSim simulator performance in comparison with the grSim simulator in headless mode for a different number of robots on field. The grSim used in the comparison had slight modifications for removing frequency limits, and it also includes a modified version with synchronous operations for comparison.

Table 1. Simulation performance in average and standart deviation of steps per second, for 1, 6 and 11 SSL robots in each team.

Simulator	1 vs 1	6 vs 6	11 vs 11
grSim (asynchronous)	2167.9 (8.4)	408.7 (0.3)	228.3 (0.1)
grSim (synchronous)	1894.0 (8.4)	390.0 (0.5)	219.0 (0.7)
rSim (proposed, synchronous)	**2408.8 (9.3)**	**510.8 (1.8)**	**288.0 (0.4)**

3.3 Environment Module

The environment module is where the environment task itself is defined. It implements the interface with the agent and communication with the other framework

[2] Code available at `https://github.com/robocin/rSim`

modules. The interface with the agent complies with the OpenAI Gym [4] framework, and it communicates with the other modules using the *entities* structures.

The use of common interfaces enables the definition of base environments, which handle the communications with the other modules and the compliance with Gym. The framework provides benchmark environments of important tasks related to the RoboCup challenges [9], serving as examples and making it easier for other researchers to develop and evaluate new RL methods in these benchmark scenarios. The work needed for defining a new environment consists of the implementation of only four methods:

- **get_commands**: Returns a list of *Robot entities* containing the commands which are sent to the simulator;
- **frame_to_observations**: Returns an observation array which will be forwarded to the agent as defined by the environment;
- **calculate_reward_and_done**: Returns both the calculated step reward and a boolean value indicating if the current state is terminal;
- **get_initial_positions_frame**: Returns a *Frame entity* used to define the initial positions of the ball and robots.

3.4 Render Module

Although we explicitly removed the graphical interface from the simulator for performance, the render module enables visualization without previous drawbacks. It renders on-demand a 2D image of the field and has no performance reduction when not in use. Its implementation is independent of the simulator and enables it to be used at training time for monitoring purposes since it is based on the Gym base environment solution.

4 Proposed Robot Soccer Environments

Due to the differences of the leagues mentioned in Section 1, we propose a complete soccer game environment based on Latin American Robotics Competition competition for the VSSS and simple skills learning environments for the SSL.

A *state* is defined as the complete set of data returned by the simulator after a performed action and an *observation* as a subset or transformation of this state. On the following proposed environments, we described the state by positions (x, y), angles (θ), and velocity $(vx, vy, v\theta)$ of each object (ball, teammate, and opponent) in reference to the field center. On the SSL environments there is an additional *Infrared sensor* (IR) signal of each robot, indicating if the ball is in contact with the kicking device.

4.1 IEEE Very Small Size League Environments

Based on Bassani et al. [3], we developed a single and a multi-agent benchmark for the VSSS league. The observation is the complete state defined above. We

describe the actions of each robot as the power percentage for each wheel that the robot will apply in the next step. For the non-controlled agents, we use a random policy based on Ornstein-Uhlenbeck process (OU) [2]. The OU process creates a more continuous motion trend for a few steps, which allows the agents to follow a more structured random trajectory instead of just oscillating around the initial point. An episode finishes if the agent received/scored a goal or if the timer reaches 30 seconds of simulation. In the **IEEE VSSS Single-Agent** environment, only one robot learns a policy, and the other five (two teammates and three opponents) follow a random policy that consists of executing actions sampled according to the OU process. In the **IEEE VSSS Multi-Agent** environment, the controlled robots share the learning policy. See on Fig. 3(a) the rendered Frame entity of the IEEE VSSS environments.

4.2 Small Size League Environments

The first environment developed is the basic GoToBall. The other environments were based on RoboCup's 2021 hardware challenge [17].

The actions of the SSL environments are the global frame velocities on each axis, kick power, and dribbler state (on/off). For all environments, we defined rewards based on energy spent by the robot, its distance to the ball, and for reaching the objective.

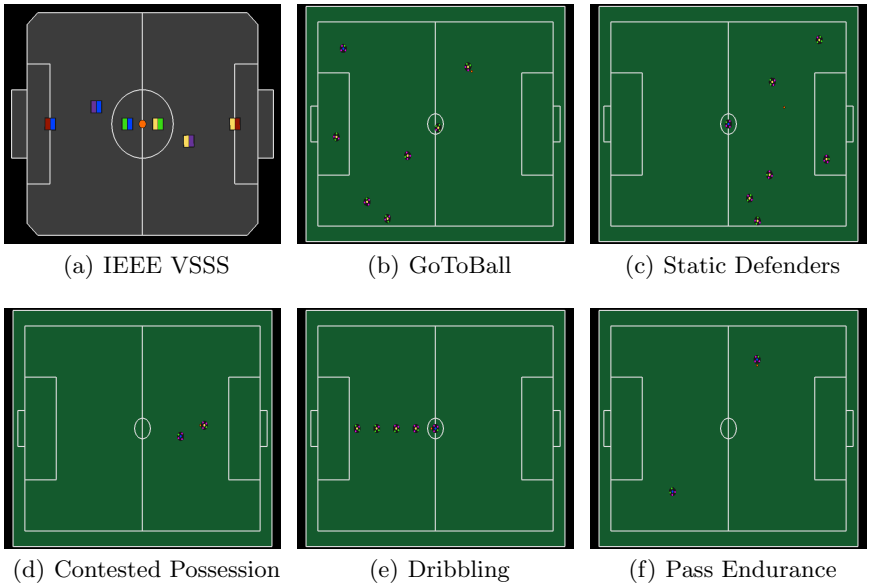
The **GoToBall** environment is the most straightforward skill to be learned. In this environment, the controlled agent must reach the ball and position its IR sensor on it, i.e., arriving at the ball at a certain angle. The episode ends when the robot completes the objective, if the agent exits the field limits, or if the simulation timer reaches 30 seconds. See on Fig. 3(b) an example of rendered Frame of the environment.

The **Hardware Challenges** environments consist of four environments based on RoboCup's 2021 hardware challenges. We made certain simplifications to the original environments to make them learnable by the currently available methods in a reasonable amount of time [17]. They are:

1. **Static Defenders:** the episode begins with the controlled agent in the field center and 6 opponents and the ball randomly positioned in opponent's field. The episode ends if the agent scores a goal, the ball or the agent exits the opponent's field, the agent collides with an opponent, or the timer reaches 30 simulated seconds. See on Fig. 3(c) an example of initial Frame.
2. **Contested Possession:** the episode begins with the controlled agent in the field center and an opponent is randomly positioned in the opponent's field, with the ball on its dribbler. The objective of this challenge is to sneak the ball from the opponent and score a goal. The episode ends with the same conditions of the Static Denfenders environment. See on Fig. 3(d) an example of initial Frame.
3. **Dribbling:** the episode begins with the controlled agent in the field center with the ball on its dribbler and four opponent robots positioned in a sparse row, leaving "gates" between each of them. The objective of this challenge is

to dribble the ball while the agent moves through these gates. The episode ends if the agent collides with any robot or exits the field. See on Fig. 3(e) an example of the initial Frame.

4. **Pass Endurance (single and multi-agent):** the episode begins with the two robots at random positions, with the ball on the dribbler of one of them. There are no opponents in this environment. In the single-agent environment, the objective is to perform a pass in three seconds. For the multi-agent, they have to perform as many passes as possible in 30 seconds. The episode ends if a pass does not reach the teammate or if the time is out. See on Fig. 3(f) an example of initial Frame.



(a) IEEE VSSS (b) GoToBall (c) Static Defenders

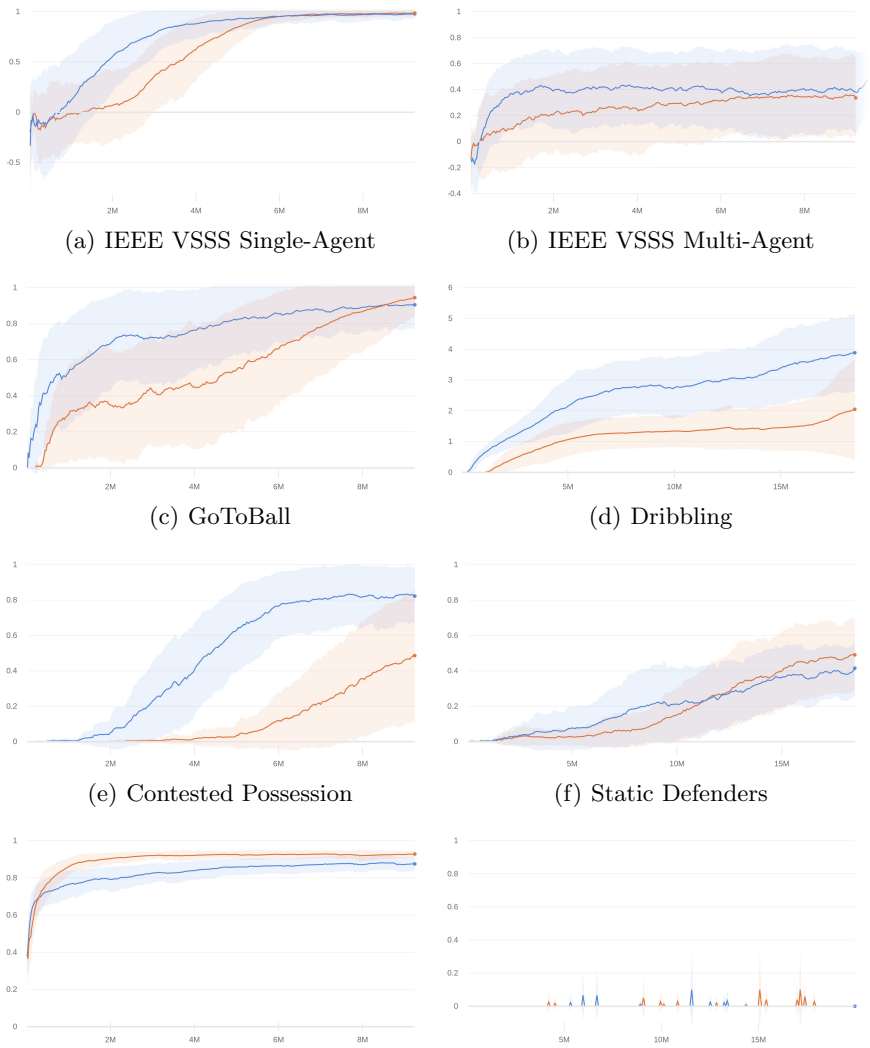
(d) Contested Possession (e) Dribbling (f) Pass Endurance

Fig. 3. Initial states of the proposed benchmark environments.

5 Experimental Results

This section presents and discusses the results obtained on our framework with two state-of-the-art deep reinforcement learning methods for continuous control.

We chose Deep Deterministic Policy Gradient (DDPG) [12] and Soft Actor Critic (SAC) [7] because both are known for presenting great performance in robot control environments such as Deepmind Control Suite [23]. We have also tested Proximal Policy Approximation (PPO) [18], however, despite all our



(a) IEEE VSSS Single-Agent (b) IEEE VSSS Multi-Agent

(c) GoToBall (d) Dribbling

(e) Contested Possession (f) Static Defenders

(g) Pass Endurance (h) Pass Endurance MA

Fig. 4. Mean (lines) and standard deviation (shades) of the results obtained for each environment (DDPG in blue and SAC in orange). The Y axis represents: Goal Score for a, b, e, and f; Ball Reached for c; Number of gates transversed for d; Inverse Distance to Receiver for g; and Pass Score for h.

efforts in parameter tuning, it was not able to learn even in the easiest environments. Therefore, we concentrated our efforts on DDPG and SAC. On the multi-agent environments, we used a shared policy to control all agents. For each environment, we executed five runs of each method. We ran 10 million steps for

each experiment, except for the Dribbling and Static Defenders environments, in which we ran 20 million steps. For the IEEE VSSS environments, Contested Possession and Static Defenders we use the goal score to evaluate the agents. In the GoToBall, and Dribbling we evaluate if the agents complete or not the respective objective. In the Pass Endurance Single-Agent we used $1/d$ to evaluate the agent, where d is the distance of the ball to the receiver. In the Pass Endurance Multi-Agent we used the pass score to evaluate the agent. In Fig. 4 we present the average and standard deviation of the learning curves obtained with each method in each environment.

We note that both algorithms presented a high standard deviation in all environments, except Pass Endurance. We also point out that DDPG was more sample efficient in most tasks (Figures 4(a) to 4(e)), an interesting result considering SAC usually performs better than DDPG in continuous control environments [7]. This performance may be explained by the fact that DDPG employs the OU process for exploration, which seems to suite better for the environments considered here. As SAC uses an entropy-based exploration, it takes more samples for it to reach the performance of DDPG, although it surpassed DDPG by a small margin at the end, in certain environments (Figures 4(c) and 4(f)).

In the multi-agent environments (IEEE VSSS and Pass Endurance), we highlight that the results were worse than the respective single-agent ones. This indicates that the agents did not learn to collaborate, since more agents were expected to perform better than a single one. For instance, in the VSSS, a visual inspection revels that, instead of collaborating, the agents block each other, as can be observed in the frame sequences available in our repository[3].

6 Conclusions and Future Work

This article presented an open-source framework for developing robot soccer RL environments for the VSSS and SSL competitions. The framework includes a simulator optimized for RL experiments and an API for defining new environments compatible with the OpenAI Gym standards. It also provides eight benchmark environments that can evaluate RL methods regarding different types of robot soccer challenges. The API is easily extensible for other types of environments and tasks. The simulator can be replaced by an interface with real robots for evaluating Sim-to-Real as in [3].

With this, we aim to put forward research and application RL methods for robot soccer by making it easier for other researchers to evaluate their strategies and compare the results in standardized scenarios, therefore improving reproducibility.

Although our results are promising in certain tasks, achieving better results than we would be able to achieve with traditional handcrafted methods, it also makes it clear that much research is needed to achieve an effective robot soccer team trained end-to-end by reinforcement learning. Studying why PPO performed so poorly is essential for our future works, once it achieved interesting

[3] `https://github.com/robocin/rSoccer`

results on other studies. The multi-agent (Figures 4(b), 4(f)) and the Static Defenders Fig. 4(h) environments show that certain benchmarks are too difficult for the currently available methods, indicating an open area of research. In the Static Defenders environment, the best reward function we developed seems inadequate when the dimensionality of the observations increases, hence the poor results. In multi-agent environments, the methods could not learn to cooperate using a shared policy. However, we believe that multi-agent specific algorithms focusing on collaboration such as MADDPG [14] would improve the results.

ACKNOWLEDGMENTS

The authors would like to thank RoboCIn - UFPE Team and Mila - Quebec Artificial Intelligence Institute for the collaboration and resources provided; Conselho Nacional de Desenvolvimento Cientifico e Tecnológico (CNPq), and Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES) for financial support. Moreover, the authors also gratefully acknowledge the support of NVIDIA Corporation with the donation of the RTX 2080 Ti GPU used for this research.

References

1. Andrychowicz, M., Baker, B., Chociej, M., Jozefowicz, R., McGrew, B., Pachocki, J., Petron, A., Plappert, M., Powell, G., Ray, A., et al.: Learning dexterous in-hand manipulation. arXiv preprint arXiv:1808.00177 (2018)
2. Arnold, L.: Stochastic differential equations. New York (1974)
3. Bassani, H.F., Delgado, R.A., de O. Lima Junior, J.N., Medeiros, H.R., Braga, P.H.M., Machado, M.G., Santos, L.H.C., Tapp, A.: A framework for studying reinforcement learning and sim-to-real in robot soccer (2020)
4. Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., Zaremba, W.: Openai gym (2016)
5. Christiano, P., Shah, Z., Mordatch, I., Schneider, J., Blackwell, T., Tobin, J., Abbeel, P., Zaremba, W.: Transfer from simulation to real world through learning deep inverse dynamics model. arXiv preprint arXiv:1610.03518 (2016)
6. Duan, Y., Liu, Q., Xu, X.: Application of reinforcement learning in robot soccer. Engineering Applications of Artificial Intelligence **20**(7), 936–950 (2007)
7. Haarnoja, T., Zhou, A., Hartikainen, K., Tucker, G., Ha, S., Tan, J., Kumar, V., Zhu, H., Gupta, A., Abbeel, P.: Soft actor-critic algorithms and applications. arXiv preprint:1812.05905 (2018)
8. Kim, J.H., Kim, D.H., Kim, Y.J., Seow, K.T.: Soccer robotics, vol. 11. Springer Science & Business Media (2004)
9. Kitano, H., Asada, M., Kuniyoshi, Y., Noda, I., Osawa, E.: Robocup: The robot world cup initiative. In: Proceedings of the First International Conference on Autonomous Agents. p. 340–347. AGENTS '97, Association for Computing Machinery, New York, NY, USA (1997). https://doi.org/10.1145/267658.267738
10. Kurach, K., Raichuk, A., Stańczyk, P., Zajac, M., Bachem, O., Espeholt, L., Riquelme, C., Vincent, D., Michalski, M., Bousquet, O., et al.: Google research football: A novel reinforcement learning environment. arXiv preprint arXiv:1907.11180 (2019)

11. Levine, S., Finn, C., Darrell, T., Abbeel, P.: End-to-end training of deep visuomotor policies. The Journal of Machine Learning Research **17**(1), 1334–1373 (2016)
12. Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., Wierstra, D.: Continuous control with deep reinforcement learning. arXiv preprint arXiv:1509.02971 (2015)
13. Liu, S., Lever, G., Merel, J., Tunyasuvunakool, S., Heess, N., Graepel, T.: Emergent coordination through competition (2019)
14. Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, O.P., Mordatch, I.: Multi-agent actor-critic for mixed cooperative-competitive environments. In: Advances in Neural Information Processing Systems. pp. 6379–6390 (2017)
15. Monajjemi, V., Koochakzadeh, A.: FIRASim. `https://github.com/fira-simurosot/FIRASim` (2020), [Online; accessed 28-April-2021]
16. Monajjemi, V., Koochakzadeh, A., Ghidary, S.S.: grsim – robocup small size robot soccer simulator. In: Röfer, T., Mayer, N.M., Savage, J., Saranlı, U. (eds.) RoboCup 2011: Robot Soccer World Cup XV. pp. 450–460. Springer Berlin Heidelberg, Berlin, Heidelberg (2012)
17. RoboCup: Robocup small size league (ssl) hardware challenges 2021. `https://robocup-ssl.github.io/ssl-hardware-challenge-rules/rules.html`, accessed: 2021-04-08
18. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347 (2017)
19. SDK, V.: VSS SDK. `https://vss-sdk.github.io/book/general.html` (2019), [Online; accessed 5-April-2021]
20. Shi, H., Lin, Z., Hwang, K.S., Yang, S., Chen, J.: An adaptive strategy selection method with reinforcement learning for robotic soccer games. IEEE Access **6**, 8376–8386 (2018)
21. Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., et al.: Mastering the game of go without human knowledge. nature **550**(7676), 354–359 (2017)
22. Sutton, R.S., Barto, A.G.: Reinforcement learning: An introduction. MIT press (2018)
23. Tassa, Y., Doron, Y., Muldal, A., Erez, T., Li, Y., de Las Casas, D., Budden, D., Abdolmaleki, A., Merel, J., Lefrancq, A., Lillicrap, T., Riedmiller, M.: Deepmind control suite (2018)
24. Vinyals, O., Babuschkin, I., Chung, J., Mathieu, M., Jaderberg, M., Czarnecki, W.M., Dudzik, A., Huang, A., Georgiev, P., Powell, R., et al.: Alphastar: Mastering the real-time strategy game starcraft ii. DeepMind Blog (2019)
25. Volodymyr, M., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I.: Playing atari with deep reinforcement learning. In: NIPS Deep Learning Workshop (2013)
26. Yoon, M.: Developing basic soccer skills using reinforcement learning for the RoboCup Small Size League. Ph.D. thesis, Stellenbosch: Stellenbosch University (2015)
27. Zhu, Y., Schwab, D., Veloso, M.: Learning primitive skills for mobile robots. In: 2019 International Conference on Robotics and Automation (ICRA). pp. 7597–7603 (2019)
28. Zolanvari, A., Shirazi, M., Menhaj, M.: A q-learning approach for controlling a robotic goalkeeper during penalty procedure. In: II International Congress on Science and Engineering. 2019. Hamburg-Germany. pp. 1–12 (2019)